

How can I check the progress of matrix multiplication?

Asked 10 years, 9 months ago Modified 1 month ago Viewed 5k times



9



I need to show an intermediate progress of matrix multiplication.

```
for(unsigned int col=0; col<mtxSize; col++) {
    unsigned tmp = 0;
    for(unsigned int row=0; row<mtxSize; row++) {
        for(unsigned int idx=0; idx<mtxSize; idx++) {
            tmp += h_A[col*mtxSize+idx] * h_B[idx*mtxSize+row];
        }
        h_Rs[col*mtxSize+row] = tmp;
        tmp = 0;
        int rate_tmp = (col*mtxSize + (row+1))*100;
        // Maybe like this...
        fprintf(stdout, "Progress : %d.%d %%\r", rate_tmp/actMtxSize,
rate_tmp%actMtxSize);
        fflush(stdout);
    }
}
```

In the case of the host code (using the CPU), it is very easy because it processes sequentially so we can check it easily.

But in the case of the GPU, which processes in parallel, what should I do?

Once the kernel is running, it does not return until the kernel execution finishes.

Thus, I can't check mid-data during the kernel execution time.

I think I need to use an asynchronous kernel call, but I do not know how.

Even if the asynchronous kernel call is used, to see all of the data into several blocks over processors, do I have to use the `atomicAdd()` (in other words, global memory access) function which includes some overhead?

I also want to know how in the case of CUDA.

[cuda](#) [Edit tags](#)

Share Edit Follow Close Flag

edited Jul 5 at 1:14

asked Dec 3, 2013 at 7:58



[L Tyrone](#)

5,511 23 28 38



[Umbrella](#)

505 3 10 20



If you only want to check the data, you still have the ability to debug the code (for cuda) with the nsight visual studio or nsight eclipse edition. For opencl there are also debuggers, too - don't know exact about it, because I develop for cuda. – [hubs](#) Dec 3, 2013 at 8:14



[hubs](#) // The data I mean was the number of element which are computed completely. Using the data I can calculate the rate of progress. – [Umbrella](#) Dec 3, 2013 at 8:18



Yes, but in the provided code you only output the rate. So I thought you'll only need it for debugging. Do you need it for following computations, too? – [hubs](#) Dec 3, 2013 at 8:30



Yes. In fact, I'm going to make progress bar using GUI. – [Umbrella](#) Dec 3, 2013 at 8:34



So for CUDA you only can split it into several kernels. For example a kernel is computing a row or several rows, because you have no influence how the sheduler works and in which order the theadblocks are computed. – [hubs](#) Dec 3, 2013 at 8:44

1 Answer

Sorted by:
Reset to default

Date modified (newest first)



Here is a code which demonstrates how to check progress from a matrix multiply kernel:

13

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define TIME_INC 100000000
#define INCS 10
#define USE_PROGRESS 1
#define MAT_DIMX 4000
#define MAT_DIMY MAT_DIMX

#define cudaCheckErrors(msg) \
do { \
    cudaError_t __err = cudaGetLastError(); \
    if (__err != cudaSuccess) { \
        fprintf(stderr, "Fatal error: %s (%s at %s:%d)\n", \
            msg, cudaGetErrorString(__err), \
            __FILE__, __LINE__); \
        fprintf(stderr, "*** FAILED - ABORTING\n"); \
        exit(1); \
    } \
} while (0)

__global__ void mykernel(volatile int *data){

    unsigned long time;
    for (int i = 0; i < INCS; i++){
        atomicAdd((int *)data,1);
        __threadfence_system();
        time = clock64();
        while((clock64() - time)<TIME_INC) {};
    }
    printf("progress check finished\n");
}

__global__ void matmult(float *a, float *b, float *c, unsigned int rowA, unsigned
int colA, unsigned int colB, volatile int *progress){
    unsigned int row = threadIdx.x+blockDim.x*blockIdx.x;
    unsigned int col = threadIdx.y+blockDim.y*blockIdx.y;
    if ((row < rowA) && (col < colB)){
        float temp = 0.0f;
        for (unsigned int k = 0; k < colA; k++)
            temp += a[(row*colA)+k] * b[(k*colB) + col];
        c[(row*colB)+col] = temp;
    #if USE_PROGRESS
        if (!(threadIdx.x || threadIdx.y)){
            atomicAdd((int *)progress, 1);
            __threadfence_system();
        }
    #endif
}

int main(){
// simple test to demonstrate reading progress data from kernel
volatile int *d_data, *h_data;
cudaSetDeviceFlags(cudaDeviceMapHost);
cudaCheckErrors("cudaSetDeviceFlags error");
cudaHostAlloc((void **)&h_data, sizeof(int), cudaHostAllocMapped);
cudaCheckErrors("cudaHostAlloc error");
cudaHostGetDevicePointer((int **)&d_data, (int *)h_data, 0);
cudaCheckErrors("cudaHostGetDevicePointer error");
*h_data = 0;
printf("kernel starting\n");
mykernel<<<1,1>>>(d_data);
cudaCheckErrors("kernel fail");
int value = 0;
do{
    int value1 = *h_data;
    if (value1 > value){
        printf("h_data = %d\n", value1);
        value = value1;}}
    while (value < (INCS-1));
    cudaDeviceSynchronize();
}
```

```

    cudaCheckErrors("kernel fail 2");

// now try matrix multiply with progress

float *h_c, *d_a, *d_b, *d_c;
h_c = (float *)malloc(MAT_DIMX*MAT_DIMY*sizeof(float));
if (h_c == NULL) {printf("malloc fail\n"); return 1;}
cudaMalloc((void **)&d_a, MAT_DIMX*MAT_DIMY*sizeof(float));
cudaCheckErrors("cudaMalloc a fail");
cudaMalloc((void **)&d_b, MAT_DIMX*MAT_DIMY*sizeof(float));
cudaCheckErrors("cudaMalloc b fail");
cudaMalloc((void **)&d_c, MAT_DIMX*MAT_DIMY*sizeof(float));
cudaCheckErrors("cudaMalloc c fail");

for (int i = 0; i < MAT_DIMX*MAT_DIMY; i++) h_c[i] = rand()/(float)RAND_MAX;
cudaMemcpy(d_a, h_c, MAT_DIMX*MAT_DIMY*sizeof(float), cudaMemcpyHostToDevice);
cudaCheckErrors("cudaMemcpy a fail");
cudaMemcpy(d_b, h_c, MAT_DIMX*MAT_DIMY*sizeof(float), cudaMemcpyHostToDevice);
cudaCheckErrors("cudaMemcpy b fail");

cudaEvent_t start, stop;
cudaEventCreate(&start); cudaEventCreate(&stop);
*h_data=0;
dim3 block(16,16);
dim3 grid(((MAT_DIMX+block.x-1)/block.x), ((MAT_DIMY+block.y-1)/block.y));
printf("matrix multiply kernel starting\n");
cudaEventRecord(start);
matmult<<<grid,block>>>(d_a, d_b, d_c, MAT_DIMY, MAT_DIMX, MAT_DIMX, d_data);
cudaEventRecord(stop);
#if USE_PROGRESS
    unsigned int num_blocks = grid.x*grid.y;
    float my_progress = 0.0f;
    value = 0;
    printf("Progress:\n");
    do{
        cudaEventQuery(stop); // may help WDDM scenario
        int value1 = *h_data;
        float kern_progress = (float)value1/(float)num_blocks;
        if ((kern_progress - my_progress)> 0.1f) {
            printf("percent complete = %2.1f\n", (kern_progress*100));
            my_progress = kern_progress;}}
        while (my_progress < 0.9f);
    printf("\n");
#endif
    cudaEventSynchronize(stop);
    cudaCheckErrors("event sync fail");
    float et;
    cudaEventElapsedTime(&et, start, stop);
    cudaCheckErrors("event elapsed time fail");
    cudaDeviceSynchronize();
    cudaCheckErrors("mat mult kernel fail");
    printf("matrix multiply finished. elapsed time = %f milliseconds\n", et);

    return 0;
}

```

The code associated with the first kernel call is just to demonstrate the basic idea of having a kernel report it's progress back.

The second part of the code shows a sample, naive matrix multiply on the GPU, with the GPU reporting it's progress back. I have included the ability to remove the progress check code via a preprocessor macro, as well as the ability to time the matrix multiply kernel. For the case I have here, there was no discernible difference in timing with or without the progress code. So while the progress reporting code probably does add *some* overhead, when compared to the scope of a reasonable sized matrix multiply kernel, it adds no significant time that I can see.

Some other uses of signalling are discussed [here](#)

Share Edit Follow Flag

edited Feb 8, 2023 at 23:00

answered Dec 4, 2013 at 17:27



Robert Crovella

150k 12 239 284

-
- ▲ Thanks Robert. I'll test my program this way. And I felt I've been using only the familiar API for me, because I have never used this two API (`cudaHostAllocMapped`, `cudaHostGetDevicePointer`). It seems that I just need to study with your code. Thanks! – [Umbrella](#) Dec 5, 2013 at 3:57
-
- ▲ The value `*h_data` never changes on my machine. With Nsight, I can see the `atomicAdd` working as the value grows in the kernel, but it is not reflected when reading in the host code. Is TCC required for this? I expected this to work on my GTX 980, but it never leaves the do-while loop. – [chappjc](#) Nov 17, 2015 at 23:48
-
- ▲ I think it works correctly on linux (just tested it again) and I think it should work similarly on windows TCC. Windows WDDM may present some challenges that I hadn't anticipated when I wrote it. Sometimes special steps are needed in WDDM mode such as described [here](#). I'm not suggesting that is an explanation for any difficulties with this code in WDDM, I merely point out that there may be some additional requirements for it to work under WDDM. I don't have a WDDM machine conveniently available ATM to test, but if time permits I'll take a look. – [Robert Crovella](#) Nov 18, 2015 at 2:11
-
- ▲ Well shoot, it works with a `cudaEventQuery`. I feel dirty doing it though. :) Thanks for the tip. Is this a bug/bugworthy issue, or simply to be expected with non-TCC or Geforce devices.? – [chappjc](#) Nov 18, 2015 at 17:44
-
- ▲ You're welcome to file bugs or requests for enhancement. It's not clear to me that it is definitely a bug. While I might like different behavior in WDDM, what is happening sort of makes sense to me. When I looked at it briefly under WDDM it did seem like it might just need a queue flush, but that seems inconsistent with your reported observation in nsight. But I haven't taken a close look. – [Robert Crovella](#) Nov 18, 2015 at 21:35
-

|